FRED TALKS

7th August 2026

CONTENTS

A PM's Guide to AI Agent Architecture: Why Capability Doesn't Equal Adoption

UMANG · 1871 WORDS

LLM Evaluation: Practical Tips at Booking.com

GEORGE CHOULIARAS · 2734 WORDS

[Daily article] August 7: Henry Macandrew

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 430 WORDS

A PM's Guide to AI Agent Architecture: Why Capability Doesn't Equal Adoption

UMANG · 05 SEP 2025 · SOURCE

Last week, I was talking to a PM who'd in the recent months shipped their AI agent. The metrics looked great: 89% accuracy, sub-second respond times, positive user feedback in surveys. But users were abandoning the agent after their first real problem, like a user with both a billing dispute and a locked account.

> "Our agent could handle routine requests perfectly, but when faced with complex issues, users would try once, get frustrated, and immediately ask for a human."

This pattern is observed across every product team that focuses on making their agents "smarter" when the real challenge is making architectural decisions that shape how users experience and begin to trust the agent.

In this post, I'm going to walk you through the different layers of AI agent architecture. How your product decisions determine whether users trust your agent or abandon it. By the end of this, you'll understand why some agents feel "magical" while others feel "frustrating" and more importantly, how PMs should architect for the magical experience.

We'll use a concrete customer support agent example throughout, so you can see exactly how each architectural choice plays out in practice. We'll also see why the counterintuitive approach to trust (hint: it's not about being right more often) actually works better for user adoption.

Let's say you're building a customer support agent

You're the PM building an agent that helps users with account issues - password resets, billing questions, plan changes. Seems straightforward, right?

But when a user says ***"I can't access my account and my subscription seems wrong"*** what should happen?

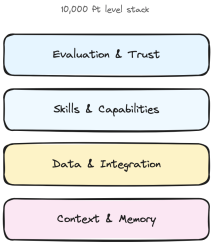
Scenario A: Your agent immediately starts checking systems. It looks up the account, identifies that the password was reset yesterday but the email never arrived, discovers a billing issue that downgraded the plan, explains exactly what happened, and offers to fix both issues with one click.

Scenario B: Your agent asks clarifying questions. "When did you last successfully log in? What error message do you see? Can you tell me more about the subscription issue?" After gathering info, it says "Let me escalate you to a human who can check your account and billing."

Same user request. Same underlying systems. Completely different products.

The Four Layers Where Your Product Decisions Live

Think of agent architecture like a stack where each layer represents a product decision you have to make.



Layer 1: Context & Memory (What does your agent remember?)

The Decision: How much should your agent remember, and for how long?

This isn't just technical storage - it's about creating the illusion of understanding. Your agent's memory determines whether it feels like talking to a robot or a knowledgeable colleague.

For our support agent: Do you store just the current conversation, or the customer's entire support history? Their product usage patterns? Previous complaints?

Types of memory to consider:

- **Session memory:** Current conversation ("You mentioned billing issues earlier...")
- **Customer memory:** Past interactions across sessions ("Last month you had a similar issue with...")
- **Behavioral memory:** Usage patterns ("I notice you typically use our mobile app...")
- **Contextual memory:** Current account state, active subscriptions, recent activity

The more your agent remembers, the more it can anticipate needs rather than just react to questions. Each layer of memory makes responses more intelligent but increases complexity and cost.

The Decision: Which systems should your agent connect to, and what level of access should it have?

The deeper your agent connects to user workflows and existing systems, the harder it becomes for users to switch. This layer determines whether you're a tool or a platform.

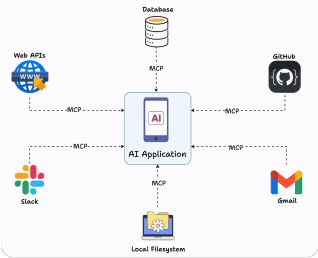
For our support agent: Should it integrate with just your Stripe's billing system, or also your Salesforce CRM, ZenDesk ticketing system , user database, and audit logs? Each integration makes the agent more useful but also creates more potential failure points - *think API rate limits, authentication challenges, and system downtime.*

Layer 3: Skills & Capabilities (What makes you different?)

The Decision: Which specific capabilities should your agent have, and how deep should they go?

Your skills layer is where you win or lose against competitors. It's not about having the most features - it's about having the right capabilities that create user dependency.

For our support agent: Should it only read account information, or should it also modify billing, reset passwords, and change plan settings? Each additional skill increases user value but also increases complexity and risk.



Implementation note: Tools like MCP (Model Context Protocol) are making it much easier to build and share skills across different agents, rather than rebuilding capabilities from scratch.

Layer 4: Evaluation & Trust (How do users know what to expect?)

The Decision: How do you measure success and communicate agent limitations to users?

This layer determines whether users develop confidence in your agent or abandon it after the first mistake. It's not just about being accurate - it's about being trustworthy.

For our support agent: Do you show confidence scores ("I'm 85% confident this will fix your issue")? Do you explain your reasoning ("I checked three systems and found...")? Do you always confirm before taking actions ("Should I reset your password now?")? Each choice affects how users perceive reliability.

Trust strategies to consider:

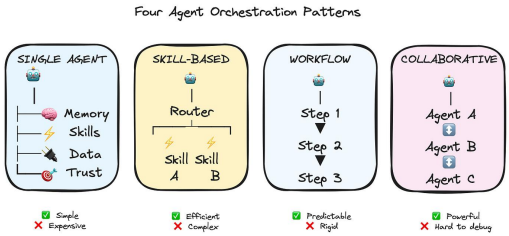
- **Confidence indicators:** "I'm confident about your account status, but let me double-check the billing details"
- **Reasoning transparency:** "I found two failed login attempts and an expired payment method"
- **Graceful boundaries:** "This looks like a complex billing issue - let me connect you with our billing specialist who has access to more tools"
- **Confirmation patterns:** When to ask permission vs. when to act and explain

> The counterintuitive insight: users trust agents more when they admit uncertainty than when they confidently make mistakes.

So how do you actually architect an agent?

Okay, so you understand the layers. Now comes the practical question that every PM asks: "How do I actually implement this? How does the agent talk to the skills? How do skills access data? How does evaluation happen while users are waiting?"

Your orchestration choice determines everything about your development experience, your debugging process, and your ability to iterate quickly.



Lets walk through the main approaches, and I'll be honest about when each one works and when it becomes a nightmare.

1. Single-Agent Architecture (Start Here)

Everything happens in one agent's context.

For our support agent: *When the user says "I can't access my account," one agent handles it all - checking account status, identifying billing issues, explaining what happened, offering solutions.*

Why this works: Simple to build, easy to debug, predictable costs. You know exactly what your agent can and can't do.

Why it doesn't: Can get expensive with complex requests since you're loading full context every time. Hard to optimize specific parts.

> Most teams start here, and honestly, many never need to move beyond it. If you're debating between this and something more complex, start here.

You have a router that figures out what the user needs, then hands off to specialized skills.

For our support agent: *Router realizes this is an account access issue and routes to the `LoginSkill`. If the LoginSkill discovers it's actually a billing problem, it hands off to `BillingSkill`.*

Real example flow:

1. *User: "I can't log in"*
2. *Router → LoginSkill*
3. *LoginSkill checks: Account exists ✓, Password correct ✗, Billing status... wait, subscription expired*
4. *LoginSkill → BillingSkill: "Handle expired subscription for user123"*
5. *BillingSkill handles renewal process*

Why this works: More efficient - you can use cheaper models for simple skills, expensive models for complex reasoning. Each skill can be optimized independently.

Why it doesn't: Coordination between skills gets tricky fast. Who decides when to hand off? How do skills share context?

> Here's where MCP really helps - it standardizes how skills expose their capabilities, so your router knows what each skill can do without manually maintaining that mapping.

You predefine step-by-step processes for common scenarios. Think LangGraph, CrewAI, AutoGen, N8N, etc.

For our support agent: *"Account access problem" triggers a workflow:*

1. *Check account status*
2. *If locked, check failed login attempts*
3. *If too many failures, check billing status*
4. *If billing issue, route to payment recovery*
5. *If not billing, route to password reset*

Why this works: Everything is predictable and auditable. Perfect for compliance-heavy industries. Easy to optimize each step.

Why it doesn't: When users have weird edge cases that don't fit your predefined workflows, you're stuck. Feels rigid to users.

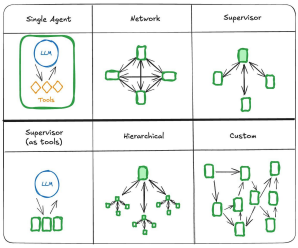
Multiple specialized agents work together using A2A (agent-to-agent) protocols.

The vision: Your agent discovers that another company's agent can help with issues, automatically establishes a secure connection, and collaborates to solve the customer's problem. *Think a booking.com agent interacting with an American Airlines agent!*

For our support agent: *`AuthenticationAgent` handles login issues, `BillingAgent` handles payment problems, `CommunicationAgent` manages user interaction. They coordinate through standardized protocols to solve complex problems.*

Reality check: This sounds amazing but introduces complexity around security, billing, trust, and reliability that most companies aren't ready for. We're still figuring out the standards.

This can produce amazing results for sophisticated scenarios, but debugging multi-agent conversations is genuinely hard. When something goes wrong, figuring out which agent made the mistake and why is like detective work.



But here's what's interesting - even with the perfect architecture, your agent can still fail if users don't trust it. That brings us to the most counterintuitive lesson about building agents.

The trust thing that everyone gets wrong

Here's something counterintuitive: Users don't trust agents that are right all the time. They trust agents that are honest about when they might be wrong.

Think about it from the user's perspective. Your support agent confidently says "I've reset your password and updated your billing address." User thinks "great!" Then they try to log in and... it doesn't work. Now they don't just have a technical problem - they have a trust problem.

Compare that to an agent that says "I think I found the issue with your account. I'm 80% confident this will fix it. I'm going to reset your password and update your billing address. If this doesn't work, I'll immediately escalate to a human who can dive deeper."

Same technical capability. Completely different user experience.

Building trusted agents requires focus on three things:

1. **Confidence calibration:** When your agent says it's 60% confident, it should be right about 60% of the time. Not 90%, not 30%. Actual 60%.

2. **Reasoning transparency:** Users want to see the agent's work. "I checked your account status (active), billing history (payment failed yesterday), and login attempts (locked after 3 failed attempts). The issue seems to be..."

3. **Graceful escalation:** When your agent hits its limits, how does it hand off? A smooth transition to a human with full context is much better than "I can't help with that."

A lot of times we obsess over making agents more accurate, when what users actually want was more transparency about the agent's limitations.

What's Coming Next

In Part 2, I'll dive deeper into the autonomy decisions that keep most PMs up at night. How much independence should you give your agent? When should it ask for permission vs forgiveness? How do you balance automation with user control?

We'll also walk through the governance concerns that actually matter in practice - not just theoretical security issues, but the real implementation challenges that can make or break your launch timeline.

LLM Evaluation: Practical Tips at Booking.com

GEORGE CHOULIARAS · 13 SEP 2025 · SOURCE

Lessons learned from 1 year of Judge-LLM Development

By

Zeno Belligoli

,

Antonio Castelli

,

George Chouliaras

The increasing adoption of Large Language Models (LLMs) across various industries has made evaluating LLM-powered applications (also called Generative AI applications) a critical necessity.

LLMs are pre-trained on a vast amount of text and can therefore be used to perform a wide range of tasks based on a natural language input called **prompt**. These tasks range from extracting information, answering complex questions, summarizing documents, generating creative content, translating languages, writing code etc.

However, unlike traditional Machine Learning (ML) models, the nature of LLMs brings inherent challenges that must be carefully considered:

- LLMs can **hallucinate**, i.e. can generate outputs which are factually incorrect or nonsensical while presenting them with high confidence
- LLMs can fail to **follow the instructions** specified in the prompt no matter how detailed they sound to a human reader
- Usage of LLMs has a **non-negligible cost** whether the model is open-source and served in-house or closed-source and accessed via a paid API.

To maximize the potential of Generative AI (GenAI) applications and mitigate the risks associated with them, we built a framework capable of thoroughly evaluating the performance of an LLM on a specific task in a nearly automated way. This framework is based on the concept of **LLM-as-judge**, i.e. on the usage of a more powerful LLM to evaluate the "target" LLM.

The LLM-as-a-judge framework

The main difference between the evaluation of traditional ML models and LLMs is that in most of the cases we are dealing with generative tasks for which either no single ground truth exists or is hard / expensive to obtain. When we ask an LLM to write a description of a property, or to summarize a long text, we don't really have an unambiguous reference that we can use for comparison. Moreover we can't get this at scale for different topics or subjects.

Therefore the measurement of text attributes like clarity, readability, toxicity or factual accuracy becomes more challenging. It would be possible, in theory, to employ human experts to review every generation and provide an estimation of the metric under study. However, this process would be time consuming and expensive to be **practically infeasible**.

The LLM-as-judge approach, requires human involvement **only once** (unless the production distribution changes), to carefully annotate the so-called **golden dataset** which must be large enough to be representative of the data distribution in production (the best practices for golden dataset creation are described in the next section).

Once we have the golden dataset with labels, we can prompt (or fine-tune) an LLM to replicate human judgement as much as possible. Once this is achieved, (e.g. when the agreement between the judge LLM and the human annotation has an accuracy above a certain threshold) the judge LLM can be employed to score the predictions of a second LLM (*target LLM*) on other datasets.

> This allows the continuous monitoring of the performance of a GenAI application in production in a scalable way with minimal human involvement.

Since the judge-LLM solves a classification task (e.g. is the text clear or not? is the text factually accurate or not?) getting golden labels is much easier. Hence, we can evaluate its performance using standard metrics like accuracy, f1-score, precision or recall, to monitor the performance of the target LLM. The process of judge-LLM development and usage is depicted in Figure 1.

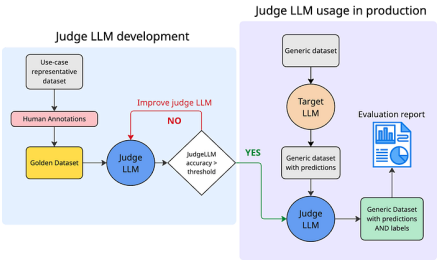


Figure 1: Judge-LLM development and deployment cycle. The left box (light-blue) shows the judge-LLM development phase. This phase always starts with a golden dataset with labels provided by a fully trusted source (e.g. human experts). A base LLM model is prompted or fine-tuned to imitate human label scoring. When the judge-LLM reaches a certain accuracy with respect to the golden labels, it can be deployed in production (right purple box) to score any other similar dataset, for example to evaluate the output of a target LLM. An evaluation report can be produced using the predictions of the target LLM and the label provided by the Judge LLM.

Golden Dataset Creation

The main aim of a golden dataset is to **accurately assess the ability** of a judge-LLM, to evaluate a production LLM in a particular evaluation metric. Due to this, a **high quality** golden dataset should meet the following criteria:

- It should represent the *production distribution*, since we want the Judge LLM evaluation to be as close to production as possible.

- It should include *golden* labels, e.g. labels with high quality. This can be proxied by high inter-annotator agreement among the annotators of the labels.

Low inter-annotator agreement usually indicates ambiguity in the metric definition and should be resolved by calibration among the annotators and by adding clarifications in the definition until the agreement is acceptable.

Since text attributes often do not have an objective and unique true value, achieving high-quality annotations is not a trivial task. To make fair comparisons between different versions of a GenAI application it is of utmost importance to have a **standardized annotation protocol** which is rigorously followed. Below we provide both a **basic** and an **advanced** annotation protocol for creating the golden dataset.

Basic Annotation Protocol

The basic annotation protocol can be used when only a single annotator is available.

1. Metric Definition

Define the metric definition with the business owner in the most unambiguous way possible.

- Write a **clear** definition of the metric as objective as possible. Prefer **binary** metrics or categorical metrics with a few classes, since LLMs struggle with continuous scores . If you're forced to use a continuous metric, consider binning into a few coarse ordinal values e.g. 'high', 'medium', 'low'.

- Write **annotation instructions**, using the metric definition. These are the instructions a human should follow in order to provide the ground truth label. Describe the task **clearly** and **objectively**. Avoid vague words (such as major, minor, a few). Provide scored examples and include edge cases. Specify what tools the annotators can and cannot use (Search engines, ChatGPT, etc).

2. Pilot Annotation

Collect a few examples (~50) and annotate them according to the aforementioned metric definition. The annotators should carefully follow the instructions provided, without bringing any previous knowledge or bias on the task. The annotators at this stage can be selected from business owners or from other technical people involved. The aim of this step is to ensure that we have a clear metric definition and that the annotations are aligned with it.

Once the annotations are done, it is recommended that they are **reviewed** by a pool of domain experts to assess if they align with the guidelines. If issues or disagreements are found, they should be resolved, the metric definition should be updated and the pilot annotation should be repeated until full consensus is reached.

3. Full Annotation by a single annotator

Once the annotation of the first small sample during the pilot annotation is finalized and reviewed, the examples can be shared to the annotator as guidance in order to annotate a full dataset. The full dataset should consist of **ideally 500- 1000 examples** at least. This is because part of the dataset will be used for tuning the judge-LLM (validation set) and another part as a holdout set in order to evaluate its performance.

4. Quality Review

A subject matter expert reviews a **representative** sample (~10%) of the full annotation. If a significant percentage (> 10%) of the labels in the sample are incorrect, annotation should be repeated. The quality thresholds can be adjusted according to the organization's needs and should be communicated and agreed upon with the annotators before the full annotation.

Advanced Annotation Protocol

The advanced annotation protocol requires more effort but also guarantees higher quality annotations than the basic one.

1. Metric Definition

Same as in the basic protocol.

2. Pilot Annotation

Same as in the basic protocol.

3. Full Annotation by multiple annotators

Instead of having a single human annotator providing the label for a given example, we have multiple human annotators (3+) providing a label for each example.
Depending on the use case, an aggregation function must be defined to obtain the final label. The aggregation function should take an array of labels and return a single one. In principle any

aggregation function can be used. If labels can be converted into integers and ordered any function like min, max, or average can be used. If labels have no orders (e.g. geographical location) majority vote can be used.

Examples where the annotators **disagree** can be treated in multiple ways, depending on what is more appropriate for the specific problem.

a) Add a "not sure" class: map to the "not sure" class all examples without 100% consensus

b) Add a weight < 1: For all examples without 100% consensus select the majority class among the annotated ones (a random one if there is no majority) and add a weight to the example equal to n_selected_class_votes / n_total_votes. This approach will penalize the more uncertain examples in the aggregated performance.

c) Discard the ambiguous examples: For some use cases it might make sense to simply remove all the examples where there is no 100% agreement.

4. Quality Review

Same as in the basic protocol.

How to develop a judge-LLM

Once the golden dataset is ready, you can develop the judge-LLM. Note that you can already do this step once the sample dataset from the pilot annotation is ready (with ~50 samples) to create a first version of the judge-LLM and score a few examples to identify any early issues. However for proper evaluation and tuning of the judge-LLM you need a large dataset of 500–1000 examples. In this article we share how to create a prompt-based judge-LLM for simplicity. You can always fine-tune a judge-LLM but this won't be covered here. To develop the judge-LLM you follow the **iterative process** detailed below, which is the standard process for manual prompt engineering.

1. **Split the golden dataset in validation and test sets:** A standard split is 50/50 but the final decision is on the model owner. The validation set is used to compute the prompt performance and to find error patterns. The test set is used to check for evaluating generalization and overfitting.

2. **Choose a strong LLM:** We typically select powerful models such as **GPT-4.1**, or **Claude 4.0 Sonnet** as the backbone for our judge-LLM. These models serve two roles:

a) As a sanity check: Given a high-quality dataset and a well-defined task, a strong LLM should perform well if the prompt is reasonable.

b) To estimate an upper bound on achievable performance. While this is not a strict bound (there are cases where smaller models with better prompts outperform stronger ones) it provides a useful reference point.

3. Write the prompt of the judge-LLM: Use the metric definition and include scoring instructions and output format (can be json/string/else). Few shot examples can also be included but ensure that no examples from the golden dataset are provided to avoid overfitting. We also normally add chain-of-thought prompting (CoT) to promote reasoning and explainability. To build the judge-LLM you can use any framework that supports custom LLM-based metrics, such as DeepEval's G-Eval metric or Arize Phoenix.

4. Evaluate the performance on the validation set: The evaluation metric depends on the type of the task and the class imbalance. For classification tasks we generally recommend macro f1-score while for continuous metric you can use mean squared error.

5. Perform error analysis and update the prompt
We analyze the model's mistakes on the validation set to identify recurring issues or edge cases. Based on these insights, we revise the prompt to better align it with the desired behavior. We then repeat steps 3, 4 and 5. Typically, we are satisfied once we reach a value of the evaluation metric above a certain threshold agreed with the business stakeholders.

6. Evaluate the performance on the test set: This will give the unbiased performance score of the judge-LLM. It is not recommended to change the prompt in order to improve the performance on the test set, as this might end up in overfitting on this dataset.

After prompt engineering is completed with a strong model, we repeat the same process using a **weaker (and more cost-efficient) model**, aiming to match the performance of the stronger version as closely as possible. In practice, we use the **strong-model judge-LLM during AI system development**, where quality is critical, and **the weaker-model version for large-scale monitoring** of system performance in production environments. An example of such monitoring is provided in Figure 2.

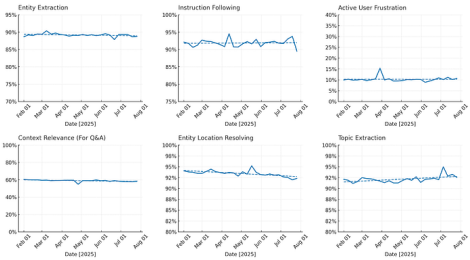


Figure 2: The figure shows an example dashboard used to monitor the quality of an LLM application, based on judge-LLM metrics. Metric values can also be combined to get aggregated scores. In this particular example we show the trends for Entity Extraction accuracy, LLM-instruction-following accuracy, frustration of users, context relevance, LLM-location-resolving accuracy and LLM-topic-conversation-extraction accuracy. From the plot it is immediate to quickly spot issues connected to instruction-following and topic-extraction allowing for prompt mitigation actions. An automated system is set-up to alert the application owners whenever an anomaly is detected for any of the relevant metrics.

Future directions

Pointwise vs. Comparative Judges

The vast majority of our judge-LLMs are **pointwise judges**, which assign an absolute score to each response independently. Pointwise judges are particularly useful because they can serve **dual purposes**:

- Ranking system outputs during development.
- Monitoring performance in production, where only one system's response is typically available.

However, **comparative (pair-wise)** judges (which compare two responses and select the better one) tend to offer **stronger ranking signals**. It is often easier for a model to decide which of two answers is better than to assign calibrated absolute scores. For this reason, comparative judges can be more effective during the AI system development phase. We are actively exploring methods to **develop both pointwise and comparative judges with minimal extra overhead**, ideally reusing most of the dataset and prompt engineering efforts.

Automatic judge-LLM development

Prompt engineering remains a **manual, iterative, and time-intensive process**, typically taking anywhere from **one day to a full week** depending on the complexity of the task. To streamline this, we have developed an **automated prompt engineering pipeline** inspired by DeepMind's OPRO that, in essence, automates **steps 3–5** of the process. At the end of the process it is always important that the human inspects the final prompt to ensure that it doesn't contain use case specific examples or rules that might undermine generalizability.

Another bottleneck in the judge-LLM development process is that of data annotation. Annotating a good dataset requires substantial effort from multiple annotators and can take from some days to weeks, depending on the complexity of the task. In this context, one can leverage the ability of LLMs to generate realistic text to create **synthetic data**. This is particularly helpful and achievable for classification tasks, whereby the LLM only needs to generate the input X, conditioned on a certain class Y. Substantial challenges in this area lie in the generation of diverse and realistic text and conversations and in the quality evaluation of the synthetic data, that is, how do we know that our synthetic data is realistic, diverse and of high-quality? We are actively working on this area and are looking forward to sharing more with you in a future blog post.

Evaluating LLM agents

We did not cover LLM-based **agent evaluation** in this article as it is a topic on its own. This is because evaluating agents often requires assessing long-term goal completion, reasoning over multi-step interactions, handling complex tool use and effective use of planning and memory. These are challenges that go beyond the evaluation of isolated model outputs and typically involve different methodologies, benchmarks, and infrastructure. We will cover this topic in a separate blog post.

Key takeaways

Evaluating generative tasks is inherently complex due to the absence of clear ground truth data. The LLM-as-a-judge framework offers an efficient method to scale this evaluation process in an automated way. A high-quality judge-LLM is built upon a "golden dataset" with reliable labels, which necessitates a rigorous annotation protocol; we have outlined both a basic and an advanced version to achieve this. This judge-LLM can then be optimized through manual, iterative prompt engineering or via automated methods. Future directions in this field include creating golden datasets with synthetic data and developing evaluation methods for LLM-based agents.

[Daily article] August 7: Henry Macandrew

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 07 AUG 2026 · SOURCE

Henry Macandrew (7 August 1866 – 16 July 1919) was a British Indian Army officer. He fought through the Tirah campaign in India, and then served as a staff officer during the Boer War, and received the Distinguished Service Order. Given command of the 5th Bengal Cavalry, upon the outbreak of the First World War he travelled to France as general staff officer grade 1 of the 1st Indian Cavalry Division. He was then promoted to become brigadier-general general staff for the Indian Cavalry Corps, with which he participated in the Battle of Neuve- Chapelle. Macandrew assumed command of the 2nd Indian Cavalry Division, leading it during the Battle of the Somme and the Battle of Cambrai. Later he was given command of the newly created 5th Cavalry Division for service in the Sinai and Palestine campaign, participating in events including the Capture of Damascus and the Battle of Aleppo. Knighted in 1919, Macandrew stayed with the 5th Cavalry Division at Aleppo, but died there later in the year. (Full article...).

Read more: https://en.wikipedia.org/wiki/Henry_Macandrew

Today's selected anniversaries:

1942:

World War II: U.S. Marines initiated the first American offensive of the Guadalcanal campaign, with landings on Tulagi, Gavutu–Tanambogo and Guadalcanal in the Solomon Islands. https://en.wikipedia.org/wiki/Battle_of_Tulagi_and_Gavutu%E2%80%93Tanambogo

1946:

The Soviet Union informed Turkey that the way in which the latter was handling the Turkish Straits no longer represented the security interests of Turkey's fellow Black Sea nations, escalating the Turkish Straits crisis. https://en.wikipedia.org/wiki/Turkish_Straits_crisis

1970:

Jonathan Jackson took hostages in a courthouse in Marin County, California, to coerce the release of the Soledad Brothers, resulting in the deaths of judge Harold Haley and three others. https://en.wikipedia.org/wiki/Marin_County_Civic_Center_attacks

2020:

Air India Express Flight 1344 overshot the runway at Calicut International Airport in Kerala, India, and crashed, killing 21 of the 190 people on board. https://en.wikipedia.org/wiki/Air_India_Express_Flight_1344

Wiktionary's word of the day:

catnip: 1. Any of the about 250 species of flowering plant of the genus Nepeta, family Lamiaceae, certain of which are said to have medicinal qualities. 2. Nepeta cataria and Nepeta grandiflora (and perhaps other species), which are well-known for causing an apparently harmless pheromone-based intoxication among certain cats. 3. (figurative) Something that causes excitement or interest. 4. About Word of the Day 5. Nominate a word 6. Leave feedback https://en.wiktionary.org/wiki/catnip

---

Wikiquote quote of the day:

Nothing you do for children is ever wasted. They seem not to notice us, hovering, averting our eyes, and they seldom offer thanks, but what we do for them is never wasted. --Garrison Keillor https://en.wikiquote.org/wiki/Garrison_Keillor